[1]mutableListOf를 각각 사용하여 다음 요구사항을 만족하도록 코드 작성하기

```
fun main() {
    // 1. 5개의 숫자 (1, 2, 3, 4, 5)를 담은 리스트를 만들고 출력
    val numbers = mutableListOf(1, 2, 3, 4, 5)
    println("1. 초기 리스트: $numbers")

    // 2. 리스트에 6을 추가한 후, 변경된 리스트를 출력
    numbers.add(6)
    println("2. 6 추가 후: $numbers")

    // 3. 리스트에서 3을 삭제한 후, 수정된 리스트를 출력
    numbers.remove(3)
    println("3. 3 삭제 후: $numbers")

    // 4. 리스트의 첫 번째 숫자를 10으로 수정한 후, 변경된 리스트를 출력
    numbers[0] = 10 // 또는 numbers.set(0, 10)
    println("4. 첫 번째 숫자 10으로 수정 후: $numbers")

    // 5. "6, 7, 8"을 리스트에 추가하고 출력
    numbers.addAll(listOf(6, 7, 8))
    println("5. 6, 7, 8 추가 후: $numbers")
}
```

```
1. 초기 리스트: [1, 2, 3, 4, 5]
2. 6 추가 후: [1, 2, 3, 4, 5, 6]
3. 3 삭제 후: [1, 2, 4, 5, 6]
4. 첫 번째 숫자 10으로 수정 후: [10, 2, 4, 5, 6]
5. 6, 7, 8 추가 후: [10, 2, 4, 5, 6, 6, 7, 8]
```

[2]associateBy() 와 groupBy() 를 비교 설명하시오.

두 함수 모두 컬렉션의 요소를 Key-Value 형태의 Map으로 변환할 때 사용하지만, 중복 키가 발생했을 때 처리 방식과 반환 타입에서 핵심적인 차이가 있습니다.

반환 타입 :

associateBy(): Map<K, V> (값 타입: 단일 객체)
groupBy(): Map<K, List<V>> (값 타입: 리스트)

중복 키 처리 :

associateBy(): 나중에 들어온 요소가 이전 요소를 덮어씀
groupBy(): 같은 키를 가진 요소들을 리스트로 그룹화

주 목적 :

associateBy(): 유일한 키(id 등)로 1:1변환/검색
groupBy(): 특정조건(그룹,카테고리)으로 1:N 분류

[3] 다음 코드에서 bar에 foo 참조를 대입할 수 있는 이유를 설명하라.

```
fun foo(x:Int, y:Int) = (x+y).toString()
val bar:Int.(Int)->String = ::foo
fun main() {
println(bar(55, 22))// 77
}
```

해설: 우선 foo(x:Int, y:Int) 이 형태와 bar:Int.(Int) 이 형태는 숫자두개를 받아서 작업을 한다는 공통점으로 코틀린 내부에선 두타입의 시그니처가 일치함, 그래서 ::foo 로 foo를 참조하라 했을 때 둘의 형태가 같아서 대입이 허용된다.

[4] Person 클래스로 person 객체를 생성 할때, apply 함수를 사용하여 한 번에 이름과 나이를 설정하는 코드를 작성하세요.

```
class Person(var name: String = "", var age: Int = 0)

fun main() {
    val person = Person().apply {
        name = "최웅"
        age = 27
    }
    println(person.name)
}
```

해설: apply를 쓰면 코드중복을 줄이고 가독성을 높임 ex)원래 person.name = “누구” person.age = 20 이렇게 쭉적을필요 없이 apply에 {}중괄호 안에 다 넣어서 가독성도 높이고 중복코드를 줄인다.

[5] Person 클래스로 person 객체를 생성하고, run 함수를 사용하여 한 번에 이름과 나이를 설정하고 마지막에 특정 문자열 메시지를 반환하는 코드를 작성하세요.

```
class Person(var name: String = "", var age: Int = 0)

fun main() {
    val person = Person()
    val message = person.run {
        name = "최웅"
        email = choiung@email.com
        age = 27

        // 마지막에 위치한 이 문자열이 message 변수에 반환됩니다.
        "이름은 ${name}이고, 이메일은 ${email}입니다."
    }
```

```
    println(message)
}
```

해설: run은 apply처럼 세팅을 시키고 맨마지막줄에 값이나 변수나 멘트나 어떤것이든 반환하는 특성 apply는 객체 자신을 결과로 내보낸다면 run은 {}맨마지막줄을 무조건 반환
예를들어 “등록이 완료되었습니다” 적으면 마지막에 저 멘트가 반환 age 이러면 27 반환 이런식으로 작동함.